

Documentação: Trabalho Prático 1

Análise Dinâmica de Carteiras de Ações

Amanda Vasconcelos Pereira
Matrícula: 2025049387

Sumário

1	Introdução	2
2	Metodologia	2
2.1	Estruturas de Dados e TADs	2
2.1.1	TAD Ação	2
2.1.2	TAD Cliente	3
2.1.3	TAD Sistema	3
2.2	Algoritmos de Ordenação	4
3	Análise de Complexidade	4
3.1	Registro de Preços (P)	4
3.2	Compra de Ação (B)	4
3.3	Venda de Ação (V)	5
3.4	Consulta de Ranking (Q)	5
3.5	Complexidade de Espaço Global	5
4	Estratégias de Robustez	6
4.1	Gerenciamento Manual de Memória	6
4.2	Tratamento de Exceções no main	6
4.3	Validação de Índices e Estados	6
4.4	Estrutura de Dados	6
5	Análise Experimental	7
5.1	Metodologia e configuração	7
5.2	Resultado dos Testes	7
5.3	Análise dos Resultados	8
5.3.1	Impacto da Escala (N)	8
5.3.2	Complexidade das Métricas e Janela (w)	8
5.3.3	Cenário Crítico: Muitas Consultas vs. Poucas Cotações	8
5.3.4	Frequência de Operações	9
5.4	Conclusão da Análise Experimental	9
6	Conclusão	9
7	Bibliografia	9

1 Introdução

Nesse trabalho foi desenvolvido um programa para a empresa *Jolambs*, focado na análise dinâmica de carteiras de ações. O objetivo principal é realizar consultas que retornem as melhores e as piores ações da carteira de um cliente.

O programa deve gerenciar dinamicamente um conjunto de ações, seu histórico de cotações e a composição da carteira dos clientes. Além de atualizar a carteira do cliente a cada compra e venda de ação.

Para o ranqueamento das ações de um cliente de acordo com o desempenho global dos ativos, o sistema deve suportar quatro métricas principais:

- **RET**: Retorno acumulado;
- **AVGRET**: Média dos retornos elementares;
- **STAB**: Estabilidade baseada na volatilidade;
- **CONS**: Consistência dos retornos positivos.

Os cálculos baseiam-se em uma janela de w cotações recentes, definindo-se os retornos elementares como: $r_i = \frac{p_i}{p_{i-1}} - 1$.

O programa deve combinar múltiplas métricas em uma única consulta através de um sistema de pontuação ponderada para ranquear as ações da carteira de um cliente.

A implementação foca na eficiência algorítmica, comparando duas estratégias distintas para a ordenação dos dados: a **Estratégia Imediata**, que atualiza as ordenações globais a cada nova cotação, e a **Estratégia Sob Demanda**, que reconstrói as ordenações apenas no momento da consulta, as quais serão detalhadas na seção 2.1.3 e comparadas na seção 5. O sistema foi desenvolvido em **C++11**, com foco em Complexidade Assintótica e gerenciamento manual de memória através de Tipos Abstratos de Dados (TADs) customizados.

2 Metodologia

O sistema foi desenvolvido em C++11, priorizando o uso de ponteiros e vetores dinâmicos em vez de bibliotecas prontas, conforme as restrições do trabalho.

2.1 Estruturas de Dados e TADs

O projeto foi modularizado em três componentes principais, cada um responsável por uma parte da lógica de negócio e do gerenciamento de dados:

2.1.1 TAD Ação

Este TAD é responsável por armazenar o identificador da ação e seu histórico de preços.

- **Armazenamento Circular**: Para otimizar o espaço e manter apenas as últimas w cotações, foi utilizado um vetor dinâmico operando como uma fila circular. Isso evita o deslocamento de elementos a cada nova inserção, garantindo que a operação de registro de preço seja $O(1)$.

- Cálculo de Métricas: Implementa as funções para os cálculos de RET, AVGRET, STAB e CONS baseando-se nas fórmulas estatísticas fornecidas. O acesso aos preços é feito através de um mapeamento de índice lógico para o índice real do vetor circular.

2.1.2 TAD Cliente

Gerencia as informações do cliente e sua posse de ativos (carteira).

- Lista Encadeada: A carteira de ações de cada cliente foi implementada utilizando uma lista encadeada de nós manual (NodoAcao). Essa escolha permite que a compra (B) e a venda (V) de ações sejam flexíveis, sem a necessidade de realocar grandes blocos de memória, já que o número de ações em uma carteira é dinâmico.

2.1.3 TAD Sistema

Atua como núcleo integrador do projeto. Ele gerencia a interação entre o conjunto global de ações e as carteiras individuais dos clientes para gerar o ranking final.

- Vetores de Ponteiros: O sistema mantém dois vetores principais: um para objetos Acao e outro para Cliente. Como os IDs são sequenciais de 0 a $N - 1$, o acesso a qualquer entidade é feito de forma direta ($O(1)$) através do índice do vetor.

1. Estratégia sob demanda

- Registro Instantâneo: O método registrarPreco apenas insere o novo valor no histórico da ação correspondente. Não há recálculo de métricas nem reordenação de rankings nesse momento.
- Gatilho de Reconstrução: Antes de entregar qualquer dado, o sistema verifica se houve novas cotações desde a última consulta. Se os dados mudaram, ele dispara a reconstrução dos rankings globais.
- Processamento JIT (Just-in-Time): Durante a consulta, o sistema:
 - (a) Varre todas as ações para calcular os valores atuais das 4 métricas (RET, AVGRET, STAB, CONS).
 - (b) Gera os rankings globais para cada métrica individualmente usando o algoritmo de ordenação (Selection Sort).
 - (c) Combina esses rankings usando os pesos fornecidos na consulta para gerar a pontuação final ponderada.
- Filtragem Local: Após gerar o ranking ponderado global, o sistema percorre a lista ligada da carteira do cliente para filtrar apenas as ações que pertencem àquele usuário específico, exibindo então as N melhores e N piores.
- Custo computacional:
 - Nova cotação (P): $O(1)$ (baixo), apenas armazena o preço no vetor da ação.
 - Consulta (Q): $O(4 \cdot N^2 + m \cdot N)$ (alto), calcula métricas, ordena rankings globais e pondera.

2. Estratégia imediata

- **Atualização antecipada:** Sempre que o método `registrarPreco` é invocado, o sistema insere a nova cotação na ação correspondente e chama imediatamente a função `atualizarTodosOsRankings()`.
- **Processamento em Cascata:** A função de atualização percorre todas as 4 métricas disponíveis (RET, AVGRET, STAB e CONS), recalcula os valores para todas as N ações do sistema e executa o algoritmo de ordenação para cada uma delas.
- **Persistência do Ranking:** O resultado dessas ordenações é armazenado na matriz `rankingsGlobais`. Isso garante que, a qualquer momento, o sistema saiba exatamente qual a posição de cada ação em cada critério de desempenho.
- **Eficiência na Consulta (Q):** No momento da consulta, o sistema não precisa calcular métricas ou ordenar as ações por métrica individual. Ele apenas acessa as posições pré-calculadas em `rankingsGlobais`, aplica os pesos da consulta para gerar a pontuação final ponderada e filtra o resultado para a carteira do cliente.
- **Custo computacional:**
 - Nova cotação (P): $O(4 \cdot N^2)$ (alto), recalcula e ordena todos os 4 rankings globais.
 - Consulta (Q): $O(m \cdot N)$ (baixo), apenas lê os rankings prontos e combina os pesos.

2.2 Algoritmos de Ordenação

Para a geração dos rankings, foi utilizado o algoritmo *Selection Sort*. Embora existam algoritmos mais rápidos, ele foi escolhido pela facilidade de implementação e por garantir a estabilidade necessária para aplicar os critérios de desempate por ID de ação exigidos pela especificação.

3 Análise de Complexidade

A análise considera N como o número total de ações, w como o tamanho da janela de cotações, u como o número de clientes, k como o tamanho da carteira do cliente e m como a quantidade de métricas em uma consulta.

3.1 Registro de Preços (P)

A operação de registrar um novo preço na classe `Acao` utiliza a lógica de vetor circular.

- **Tempo:** $O(1)$, a inserção ocorre em uma posição direta do vetor usando `indiceAtual`, sem necessidade de deslocar elementos.
- **Espaço:** $O(w)$ por ação, para armazenar o histórico de preços solicitado.

3.2 Compra de Ação (B)

A operação de compra de uma ação manipula a lista encadeada da carteira do cliente.

- **Tempo:** $O(1)$, como a carteira do cliente é uma lista encadeada e a inserção é feita sempre no início (head), a operação não depende do tamanho da carteira.
- **Espaço:** $O(1)$ por operação, é alocado apenas um novo nó na memória para armazenar o identificador da ação comprada.

3.3 Venda de Ação (V)

A operação de venda de uma ação manipula a lista encadeada da carteira do cliente.

- **Tempo:** $O(K)$ no pior caso, quando a ação a ser removida é a última da lista encadeada ou não existe, exigindo que o sistema percorra todos os K elementos da carteira do cliente.
- **Espaço:** $O(1)$, a operação apenas libera a memória de um nó existente, não exigindo alocação de espaço adicional.

3.4 Consulta de Ranking (Q)

A consulta é a operação mais complexa, pois envolve o processamento de métricas e ordenações (análise feita sobre a estratégia de ordenação sob demanda).

- **Tempo:** $O(m \cdot N^2 + N \cdot K + m \cdot N \cdot w)$
 1. **Cálculo de métricas:** Para cada métrica (m), calcula-se o valor para todas as ações (N), processando a janela (w). Custo: $O(m \cdot N \cdot w)$.
 2. **Ordenação:** O *Selection Sort* é executado m vezes para gerar os rankings parciais e uma vez para o ranking final. Custo: $O(m \cdot N^2)$.
 3. **Filtragem:** Para projetar o ranking global na carteira local, percorre-se o ranking (N) e, para cada item, verifica-se a presença na lista encadeada do cliente (K). Custo: $O(N \cdot K)$.
- **Espaço:** $O(N + m + k)$, durante a execução, são alocados vetores temporários para armazenar as pontuações do ranking global (N), as métricas da consulta (m) e o resultado filtrado da carteira do cliente.

3.5 Complexidade de Espaço Global

O sistema ocupa memória de forma proporcional às entidades declaradas:

- **Ações:** $O(N \cdot w)$ para armazenar os históricos.
- **Clientes:** $O(u + \sum K_i)$, onde u é o vetor de clientes e $\sum K_i$ é o somatório de todos os nós de todas as listas encadeadas (total de ações em posse de clientes).
- **Total:** $O(N \cdot w + u + L)$, onde $N \cdot w$ representa o histórico das ações, u o vetor de clientes e L o total de nós de listas encadeadas ativos no sistema.

4 Estratégias de Robustez

4.1 Gerenciamento Manual de Memória

Para garantir que tudo que foi alocado com `new` seja liberado com `delete`, foram implementados:

- **Destrutores:** Os TADs `Acao`, `Cliente` e `Sistema` possuem destrutores que limpam arrays dinâmicos e listas encadeadas;
- **Prevenção de vazamento em reinicialização:** No método `Acao::inicializar`, o código verifica se `historico` já aponta para algo antes de alocar um novo array, evitando perder o endereço da memória antiga.

4.2 Tratamento de Exceções no main

Foi usado um bloco `try-catch` para envolver toda a execução principal (`main`). O programa possui um `std::bad_alloc`, que captura falhas de alocação de memória, e um `std::exception`, que captura qualquer erro inesperado na biblioteca padrão, impedindo que o programa pare sem explicação.

4.3 Validação de Índices e Estados

O código verifica se os dados não foram inseridos de forma errada ou incompletos.

- **Verificação de limites:** No `Sistema.cpp`, os métodos `registrarPreco`, `comprarAcao` e `venderAcao` verificam se os IDs fornecidos estão dentro do intervalo válido (0 a $nAcoes - 1$ ou $nCliente - 1$) antes de acessar os arrays;
- **Divisão por zero:** No cálculo de retorno e métricas, existe uma checagem para evitar divisões por zero;
- **Estado mínimo para cálculos:** As funções de métrica usam o método `temCotacoesSuficientes` para garantir que o cálculo só ocorra se o histórico estiver cheio (tamanho w), retornando 0.0 caso contrário.

4.4 Estrutura de Dados

- **Vetor circular:** A implementação do histórico de preços como um vetor circular ($indiceAtual = (indiceAtual + 1) \% w$) é uma estratégia robusta para manter apenas as w cotações mais recentes sem precisar deslocar todos os elementos do array a cada inserção, o que evita erros de lógica e melhora a performance;
- **Tratamento de listas encadeadas:** No `Cliente::removerAcao`, há o cuidado de tratar tanto a remoção do primeiro elemento (ajustando o `head`) quanto de elementos no meio/fim, garantindo a integridade da lista da carteira.

5 Análise Experimental

A análise experimental busca avaliar o desempenho das estratégias Imediata e Sob Demanda sob diferentes cargas de trabalho, focando no custo de manutenção das ordenações globais.

5.1 Metodologia e configuração

Foram realizados 6 experimentos variando o número de ações (N), clientes (u), tamanho da janela (w), quantidade de métricas (m) e a frequência de operações de cotação e consulta. Os tempos foram coletados em milissegundos (ms) para o total e microssegundos (μs) para médias por operação.

Tabela 1: Descrição dos Cenários de Simulação

Cenário	Descrição
N_Baixo	Pequena escala ($N = 50, u = 10, w = 10, m = 2$).
N_Alto	Alta escala de ações, poucas consultas ($N = 1000, u = 10, w = 10, m = 2$).
Muitas_Consultas	Foco em operações de consulta (Q) com volume médio de ações ($N = 500, u = 50, w = 10, m = 2$).
Muitas_Cotacoes	Foco em processamento de preços (P) ($N = 500, u = 50, w = 10, m = 2$).
W_Longo_M_Alto	Janela extensa e mais métricas ($N = 500, u = 50, w = 100, m = 4$).
Muitas_Consultas_Poucas_Cotacoes	Alta frequência de consultas em relação às atualizações de preço ($N = 500, u = 50, w = 10, m = 4$).

5.2 Resultado dos Testes

Demanda					
Cenario	Tempo_Total_ms	Media_Cotacao_P_us	Media_Carteira_BV_us	Media_Consulta_Q_ms	Total_Reconstrucoes
N_Baixo	21.49	0.0033	0.0081	0.1188	140
N_Alto	609.45	0.0241	0.0789	82.484	130
Muitas_Consultas	613.3	0.041	0.4	24.377	442
Muitas_Cotacoes	110.51	0.0213	0.1967	22.571	70
W_Longo_M_Alto	822.14	0.0439	0.0973	102.651	212
Muitas_Consultas_Poucas_Cotacoes	827.19	0.0094	0.0357	30.002	1044

Figura 1: Resultados da Estratégia Sob Demanda

Imediata					
Cenario	Tempo_Total_ms	Media_Cotacao_P_us	Media_Carteira_BV_us	Media_Consulta_Q_ms	Total_Reconstrucoes
N_Baixo	56.43	429.706	0.0846	0.0255	116
N_Alto	81055.61	79.298.695	0.1947	15.508	122
Muitas_Consultas	10554.59	20.361.128	0.0152	0.404	418
Muitas_Cotacoes	11024.87	20.692.055	0.0806	0.4474	70
W_Longo_M_Alto	107185.63	20.918.114	0.123	0.403	264
Muitas_Consultas_Poucas_Cotacoes	10585.01	20.423.002	0.0	0.4031	1052

Figura 2: Resultados da Estratégia Imediata

5.3 Análise dos Resultados

5.3.1 Impacto da Escala (N)

Nos cenários N_Baixo e N_Alto, observamos a maior discrepância de desempenho. Na estratégia Imediata, o tempo total saltou de 56,43ms para 81.055,61ms quando N subiu de 50 para 1000, um aumento de aproximadamente 1.436 vezes. Isso ocorre porque o custo médio de processar uma cotação (Media_Cotacao_P_us) na estratégia Imediata é extremamente sensível a N , atingindo 79.298,69μs no cenário alto. Em contrapartida, a estratégia Sob Demanda saltou de 21,49ms para 609,45ms no total, um aumento de aproximadamente 28 vezes, pois "adia" o custo da ordenação apenas para o momento da consulta.

5.3.2 Complexidade das Métricas e Janela (w)

O cenário W_Longo_M_Alto ($w = 100$, $m = 4$) gerou o maior tempo total na estratégia Imediata: 107.185,63ms. O aumento de w torna o cálculo das métricas (como STAB e AVGRET) mais difícil e o aumento de m exige a manutenção de mais estruturas de ordenação global simultâneas. Na estratégia Sob Demanda, o tempo foi de apenas 822,14ms, evidenciando que processar cotações sem reordenar o sistema é drasticamente mais eficiente.

5.3.3 Cenário Crítico: Muitas Consultas vs. Poucas Cotações

O teste Muitas_Consultas_Poucas_Cotacoes foi feito para desafiar a estratégia Sob Demanda, apresentando o maior número de reconstruções registradas (1044).

- **Estratégia imediata:** Apresentou um tempo total de 10.585,01ms. O custo por consulta foi baixo (0,4031ms), pois o ranking já estava pronto.
- **Estratégia sob demanda:** Mesmo com 1044 reconstruções de ranking, o tempo total foi de 827,19ms.

Embora o tempo médio de consulta (Media_Consulta_Q_ms) na Sob Demanda seja maior que na Imediata (30,00ms vs 0,40ms), a economia gerada ao não reordenar o sistema a cada nova cotação (que na Imediata custa 20.423,00μs por operação) compensa amplamente o esforço de reconstrução.

5.3.4 Frequência de Operações

Comparando Muitas_Consultas e Muitas_Cotacoes na estratégia Sob Demanda, nota-se que o tempo total cai de 613,3ms para 110,51ms quando o volume de consultas diminui. Isso confirma que a estratégia Sob Demanda é otimizada para sistemas onde o fluxo de dados (cotações) é mais intenso que a demanda por relatórios (consultas).

5.4 Conclusão da Análise Experimental

Os dados demonstram que a Estratégia Sob Demanda é a mais eficiente para os volumes testados. O custo de manter uma estrutura global sempre ordenada (Imediata) escala muito rápido com o aumento de N e m . A estratégia Imediata só seria justificável em cenários de baixíssima latência para a resposta final (Q), onde o tempo de 30ms a 100ms por consulta da Sob Demanda fosse inaceitável, e o número de ações fosse reduzido o suficiente para não comprometer o processamento das cotações.

6 Conclusão

Este trabalho prático permitiu a implementação de um sistema completo de análise de carteiras de ações, aplicando conceitos fundamentais de Tipos Abstratos de Dados (TADs) e gerenciamento dinâmico de memória em C++11. Através da análise experimental, conclui-se que a Estratégia Sob Demanda é amplamente superior para o perfil de uso da maioria dos sistemas financeiros modernos, onde a frequência de atualização de preços (cotações) é muito superior à frequência de consultas de ranking pelos clientes.

O desenvolvimento reforçou o aprendizado sobre a importância da escolha de estruturas de dados adequadas, como o uso de vetores circulares para janelas de tempo fixas ($O(1)$ na inserção) e listas encadeadas para carteiras de tamanho variável. Além disso, a análise assintótica comprovou que, embora o custo de ordenação seja inevitável, adiar esse processamento evita cálculos redundantes e otimiza o tempo total de execução do sistema em cenários de alta volatilidade de dados.

7 Bibliografia

- ANISIO; GISELE; MÁRCIO; WAGNER. Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações. DCC221 – Estruturas de Dados. Belo Horizonte: Departamento de Ciência da Computação, Universidade Federal de Minas Gerais (UFMG), 2026.
- ZIVIANI, Nivio. Projeto de Algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.
- Slides da disciplina Estruturas de Dados - DCC/UFMG.
- GOOGLE. Gemini (versão Gemini 3 Flash). [Software de inteligência artificial].
- ANTHROPIC. Claude (versão 3.5 Sonnet). [Software de inteligência artificial].
- ZIVIANI, Nivio. Projeto de Algoritmos: com implementações em Java e C++. São Paulo: Cengage Learning, 2007.